

Documentación de Integración Stripe - OmniTech

1. Configuración de Stripe

1.1 Instalación del paquete

```
pip install stripe
```

1.2 Configuración en settings.py

Archivo: C:\Users\grosa\OneDrive\Escritorio\Proyecto\omnitech\settings.py (líneas 110-112)

```
STRIPE_PUBLIC_KEY = 'pk_test_...'  
STRIPE_SECRET_KEY = 'sk_test_...'  
STRIPE_CURRENCY = 'clp'
```

- STRIPE_PUBLIC_KEY : Clave pública para usar en el frontend (Stripe.js)
 - STRIPE_SECRET_KEY : Clave secreta para usar en el backend
 - STRIPE_CURRENCY : Moneda (clp = pesos chilenos)
-

2. Servicio de Stripe

2.1 Archivo:

C:\Users\grosa\OneDrive\Escritorio\Proyecto\omnitech\stripe_service.py

Este archivo contiene toda la lógica de comunicación con la API de Stripe.

2.1.1 Configuración inicial (líneas 1-7)

```
import stripe
from django.conf import settings
from decimal import Decimal

stripe.api_key = settings.STRIPE_SECRET_KEY
```

2.1.2 Función crear_checkout_session() (líneas 11-69)

Esta función crea una sesión de pago en Stripe.

Parámetros: - `order` : Objeto Order de Django - `items` : Lista de productos del carrito - `success_url` : URL a la que redirige si el pago es exitoso - `cancel_url` : URL a la que redirige si el pago se cancela

Proceso: 1. Convierte los productos del carrito en `line_items` de Stripe 2. Agrega el costo de envío como un item adicional 3. Crea la sesión de Checkout con `stripe.checkout.Session.create()`

Código clave:

```
checkout_session = stripe.checkout.Session.create(
    payment_method_types=['card'],
    line_items=line_items,
    mode='payment',
    success_url=success_url,
    cancel_url=cancel_url,
    customer_email=order.email_invitado or (order.usuario.email if order.usuario else None),
    metadata={'order_id': order.numero_pedido},
)
```

2.1.3 Cálculo de montos (líneas 22-30)

```
if moneda == 'clp':
    unit_amount = int(float(item['precio'])) # CLP no usa decimales
else:
    unit_amount = int(float(item['precio']) * 100) # USD/EUR usan centavos
```

3. Vistas de Django

3.1 Archivo:

C:\Users\grosa\OneDrive\Escritorio\Proyecto\omnitech\views.py

3.1.1 Importaciones (líneas 1-26)

```
from .stripe_service import crear_checkout_session
```

3.1.2 Vista checkout() (líneas 288-313)

Muestra la página de checkout con el formulario.

Contexto enviado: - carrito : Items del carrito - subtotal , costo_envio , total : Montos - stripe_public_key : Para usar Stripe.js en el frontend

3.1.3 Vista crear_sesion_stripe() (líneas 316-361)

Esta es la vista que procesa el formulario y crea la sesión de Stripe.

Flujo: 1. Obtiene los datos del formulario (email, región, observaciones) 2. Crea un pedido en estado PENDIENTE_PAGO 3. Guarda el carrito temporalmente en la sesión 4. Llama a crear_checkout_session() para obtener la URL de Stripe 5. **Redirige al usuario a la URL de Stripe** (return redirect(session.url))

Código clave:

```
# Crear pedido pendiente
order = Order.objects.create(
    numero_pedido=numero_pedido,
    estado=OrderState.PENDIENTE_PAGO,
    ...
)

# Guardar carrito en sesión
request.session['order_id'] = numero_pedido
request.session['carrito_temp'] = carrito
```

```
# Crear sesión de Stripe
success_url = request.build_absolute_uri('/pago-exitoso/?session_id={{CHECKOUT_?
cancel_url = request.build_absolute_uri('/checkout/')
session = crear_checkout_session(order, totales['items'], success_url, cancel_ur

# Redirigir a Stripe
return redirect(session.url)
```

3.1.4 Vista pago_exitoso() (líneas 363-423)

Esta vista procesa el pago después de que Stripe confirma el pago.

Flujo: 1. Obtiene el `session_id` de Stripe desde la URL 2. Recupera el pedido guardado en la sesión 3. Si el pedido está en estado `PENDIENTE_PAGO` : - Decrementa el stock de productos - Marca las licencias como consumidas - Crea los OrderItems - Actualiza el estado a `PAGADO_PROCESANDO` - Envía la boleta por email 4. Renderiza la página de confirmación

Código clave:

```
# Verificar que el pago fue exitoso
if session.payment_status == 'paid':
    # Procesar el pedido
    for item in carrito_temp:
        if tipo == 'fisico':
            producto.stock_fisico -= cantidad
            OrderItem.objects.create(pedido=order, producto_fisico=producto, ..
        else:
            licencia.estado_licencia = LicenseState.CONSUMIDA
            OrderItem.objects.create(pedido=order, licencia=licencia, ...)

# Actualizar estado
order.estado = OrderState.PAGADO_PROCESANDO

# Enviar boleta
enviar_boleta_pedido(order)
```

4. URLs

4.1 Archivo:

C:\Users\grosa\OneDrive\Escritorio\Proyecto\omnitech\urls.py

```
path('crear-sesion-stripe/', views.crear_sesion_stripe, name='crear_sesion_stripe')
path('pago-exitoso/', views.pago_exitoso, name='pago_exitoso'),
```



5. Template de Checkout

5.1 Archivo:

C:\Users\grosa\OneDrive\Escritorio\Proyecto\templates\checkout.html

5.1.1 Estructura del formulario (líneas 340-490)

```
<form method="POST" action="{% url 'crear_sesion_stripe' %}">
    {% csrf_token %}

    <!-- Información de Contacto -->
    <input type="email" name="email" required>
    <input type="tel" name="telefono">

    <!-- Dirección de Envío -->
    <select name="region" id="region">...</select>
    <select name="ciudad" id="ciudad">...</select>
    <input type="text" name="direccion">

    <!-- Método de Pago -->
    <input type="radio" name="pago" value="stripe" checked>

    <button type="submit">Pagar con Stripe</button>
</form>
```

5.1.2 JavaScript para ciudades (líneas 495-570)

El JavaScript maneja: - Actualización dinámica de ciudades según la región - Cálculo del costo de envío - Selección visual de método de pago

6. Servicio de Email (Boletas)

6.1 Archivo:

C:\Users\grossa\OneDrive\Escritorio\Proyecto\omnitech\services.py

6.1.1 Función `enviar_boleta_pedido()` (líneas 84-127)

Esta función envía la boleta por email después de un pago exitoso.

Flujo: 1. Genera el HTML de la boleta 2. Envía el email con `send_mail()` de Django 3. Incluye todos los productos, precios y datos del cliente

7. Modelos Relacionados

7.1 Order (Pedido)

```
class Order(models.Model):
    numero_pedido = CharField(max_length=20, unique=True)
    usuario = ForeignKey(User, null=True)
    email_invitado = EmailField(null=True)
    estado = CharField(choices=OrderState.choices)
    subtotal = DecimalField()
    costo_envio = DecimalField()
    total = DecimalField()
    region_envio = CharField(max_length=100)
    fecha_pago = DateTimeField(null=True)
```

7.2 OrderItem (Item del Pedido)

```
class OrderItem(models.Model):
    pedido = ForeignKey(Order)
    producto_fisico = ForeignKey(PhysicalProduct, null=True)
    licencia = ForeignKey(DigitalLicense, null=True)
    cantidad = PositiveIntegerField()
    precio_unitario = DecimalField()
```

8. Flujo Completo de Pago

FLUJO DE PAGO

1. USUARIO VE PRODUCTOS

↳ Home → Productos → Detalle → Agregar al Carrito

2. USUARIO VA AL CHECKOUT

↳ /checkout/

↳ Vista: checkout()

↳ Renderiza: checkout.html

3. USUARIO LLENA FORMULARIO

↳ Email, Región, Ciudad, Dirección

↳ Click en "Pagar con Stripe"

4. SE CREA SESIÓN EN STRIPE

↳ Vista: crear_sesion_stripe() [POST]

↳ 1. Crea Order en estado PENDIENTE_PAGO

↳ 2. Guarda carrito en sesión

↳ 3. Llama a stripe_service.crear_checkout_session()

↳ 4. Obtiene URL de Stripe Checkout

↳ 5. REDIRIGE a Stripe

5. USUARIO PAGA EN STRIPE

↳ Stripe muestra formulario de tarjeta

- ↳ Stripe procesa el pago
- ↳ Stripe redirige a /pago-exitoso/?session_id=xxx

6. SE PROCESA EL PAGO EXITOSO

- ↳ Vista: pago_exitoso()
- ↳ 1. Verifica el pago con Stripe
- ↳ 2. Recupera Order de la sesión
- ↳ 3. Decrementa stock de productos
- ↳ 4. Marca licencias como consumidas
- ↳ 5. Crea OrderItems
- ↳ 6. Actualiza estado a PAGADO_PROCESANDO
- ↳ 7. Envía boleta por email
- ↳ 8. Renderiza página de confirmación

7. USUARIO RECIBE CONFIRMACIÓN

- ↳ Página de pago_exitoso.html
- ↳ Email con boleta en su correo

9. Claves de Prueba (Test)

Para probar sin hacer pagos reales, Stripe proporciona tarjetas de prueba:

Número de Tarjeta	Resultado
4242 4242 4242 4242	Pago exitoso
4000 0000 0000 0002	Pago rechazado
4000 0025 0000 3155	Requiere autenticación 3D Secure

Ejemplo de uso: 1. Ir a Stripe Checkout 2. Ingresar: 4242 4242 4242 4242 3. MM/AA: cualquier fecha futura 4. CVC: 123 5. Código postal: 12345

10. Cambios para Producción

Para pasar a producción, cambiar en `settings.py` :

```
# Development (prueba)
STRIPE_PUBLIC_KEY = 'pk_test_...'
STRIPE_SECRET_KEY = 'sk_test_...'

# Production (real)
STRIPE_PUBLIC_KEY = 'pk_live_...'
STRIPE_SECRET_KEY = 'sk_live_...'
```

Para obtener las claves reales: 1. Ir a <https://dashboard.stripe.com> 2. Crear cuenta o iniciar sesión 3. Ir a Developers > API Keys 4. Copiar las claves `pk_live` y `sk_live`

11. Estructura de Archivos

```
C:\Users\grosa\OneDrive\Escritorio\Proyecto\
├── omnitech\
│   ├── settings.py          # Configuración de Stripe
│   ├── views.py            # Vistas de checkout y pago exitoso
│   ├── stripe_service.py   # Servicio de integración Stripe
│   ├── services.py         # Envío de boletas por email
│   ├── urls.py             # URLs del proyecto
│   └── models.py           # Modelos Order, OrderItem
├── templates\
│   └── checkout.html       # Template del checkout
├── static\css\style.css    # Estilos
└── requirements.txt        # Dependencias (incluye stripe)
```

12. Resumen de Líneas de Código Clave

Archivo	Líneas	Descripción
settings.py	110-112	Configuración de claves Stripe

Archivo	Líneas	Descripción
stripe_service.py	11-69	Creación de sesión de pago
stripe_service.py	22-30	Cálculo de montos
views.py	316-361	Vista crear_sesion_stripe
views.py	363-423	Vista pago_exitoso
checkout.html	340-490	Formulario de checkout
checkout.html	495-570	JavaScript para regiones/ciudades
services.py	84-127	Envío de boletas
urls.py	18-19	Rutas de Stripe

13. Troubleshooting

Error: "No se puede crear sesión"

- Verificar que las claves de Stripe sean válidas
- Verificar que la cuenta de Stripe esté activa

Error: "Payment failed"

- Usar tarjetas de prueba (no reales en modo test)
- Verificar que el monto no supere límites

Error: "Webhook no recibido"

- El flujo actual NO usa webhooks
- Usa redirect en vez de webhook para simplicidad

El carrito no se procesa

- Verificar que la sesión tenga `order_id` y `carrito_temp`
 - Verificar que el pedido exista en la base de datos
-

Documentación generada para OmniTech ERP & E-Commerce Integración Stripe - Abril 2026